

SAT ranges for spent transaction outputs ▾

Hey i need to find sat ranges of spent transacriion outputs

Uncovering sat range indexing flags for spent outputs >

I can help with this. Before I give you specifics, let me check the current state of the tooling since the Ordinals software changes fairly often.

Synthesized indexing requirements and API response behavior >

For spent outputs specifically, the catch is that the default sat index prunes them, so you have to opt in to keeping them.

Using `ord` (the reference indexer), the workflow is:

1. **Build the index with the right flags.** `--index-sats` gives you sat ranges, but on its own it discards ranges once an output is spent (ord prunes spent UTXOs as it forward-carries ranges block to block). To keep spent outputs you also need `--index-spent-sats`:

```
ord --index-sats --index-spent-sats index update
```

This is a much larger index, and you generally have to build it from scratch — adding the flag to an existing `--index-sats` database won't backfill the spent ranges you already pruned.

2. **Query the output.** Once indexed, hit the output endpoint:

```
curl -s -H "Accept: application/json" \
  http://0.0.0.0/output/<txid>:<vout>
```

The response includes a `sat_ranges` field plus `"spent": true/false`. Without `--index-spent-sats`, a spent output comes back with `"sat_ranges": null` and `"spent": true` — which is exactly the gap you're describing. [Ordinals](#)

A couple of things worth knowing:

- **Range format is half-open:** `[start, end)` — start inclusive, end exclusive, so the range `[1964333750000000, 1964333750000001]` only includes the single sat 1964333750000000. [Ordiscan](#)
- **The model is "forward carrying":** for each transaction in each new block, ord prunes the spent UTXOs and updates the new outputs with the carried-over ranges. That's why reconstructing spent ranges after the fact isn't possible without having retained them. [Hiro](#)

If you'd rather not run a full sat-indexed node, hosted APIs (Ordiscan, Hiro) expose sat ranges per address/output, though their handling of *spent* outputs varies.

Two quick questions so I point you the right way: are you trying to trace a **specific** sat/inscription through spends, or bulk-extract spent-output ranges for analysis? And are you running your own `ord` node or hoping to use an API?



Good catch — and that absence tells us something specific. Let me confirm the details.

2026-06-03, 14:13

This is a useful clue. The flag exists in upstream `ord` — `--index-spent-sats` was added in **0.16.0** ("Add option to retain sat index for spent outputs (#2999)"). So if it's missing from your `--help`, your binary is either older than that or a fork that doesn't track upstream. [New Releases](#)

But here's what catches my eye: your help output *does* show `--index-addresses` ("Track unspent output addresses"), which landed **later** than `--index-spent-sats` in upstream. Having the newer flag but not the older one is the signature of a **fork or a pinned build**, not just an old version — and the `counterparty-core` tab next to your `~/ord/target` tab makes me think this is Counterparty's bundled `ord`. Let me confirm that.

Weighing diagnostic evidence and formulating actionable troubleshooting steps >

ⓘ Claude's response was interrupted.

Try again

Nope i have 0.27.1

Pinpointing PR #3893 as culprit for flag removal >

0.27.1 changes everything I just said — that's newer than 0.16, so this isn't an old build or a fork missing the flag. If a current version doesn't have it, the flag was most likely **removed** at some point. Let me verify.

Traced changelog clues to pinpoint version-specific flag removal >

The likely culprit is in the changelog: a PR titled "Make index settings harder to misuse (#3893 by casey)". That reworked how the `--index-*` flags behave. Let me pull the actual changelog to confirm what happened to `--index-spent-sats` and what replaced it. [GitHub](#)

Hunting for ordinals documentation and current flags >

Now I can see exactly what happened. `--index-spent-sats` was added in 0.16.0, but **0.20.0** included "Make index settings harder to misuse (#3893 by casey)", alongside a major index restructuring — a unified `OUTPOINT_TO_UTXO_ENTRY` table (#3915) and, in 0.20.1, "Keep sat ranges in low-level format (#3963)". That overhaul is what changed the flag. Let me confirm precisely what it became so I give you the right command. [GitHub + 2](#)

Verifying absence of `index_spent_sats` setting >

The PR page didn't render usefully. Let me check whether the setting still exists anywhere in current `ord` — config file or code — rather than guess.

Traced feature removal across `ord` versions and formulated test strategy >

Mystery solved — and I need to correct what I said earlier. It's not a fork, and it's not too old. It's the opposite: the dedicated option was **removed**, and you're on a version that postdates the removal.

Here's the actual timeline:

- `--index-spent-sats` was added in **0.16.0** (#2999) and existed through the 0.19.x line.
- In **0.20.0**, Casey reworked the entire index — "Make index settings harder to misuse" (#3893) plus a unified `OUTPOINT_TO_UTXO_ENTRY` table (#3915) and keeping sat ranges in low-level format (#3963). The standalone spent-sats option didn't survive that overhaul. [GitHub + 2](#)

And it's not hiding in the config file either. The current handbook's full example config lists `index_addresses`, `index_cache_size`, `index_runes`, `index_sats`, and `index_transactions` — no `index_spent_sats`. So on 0.27.1 there's genuinely no switch to flip, CLI or YAML. [Ordinals](#)

So your options now:

1. Test whether plain `--index-sats` already retains them. The storage overhaul may have changed default behavior. Fastest way to know is empirical — point your sat-indexed server at a *known-spent* outpoint and check the field:

```
curl -s -H "Accept: application/json" http://0.0.0.0/output/
<spent_txid>:<vout>
```

If `sat_ranges` is populated (not `null`) while `spent` is `true`, you're done. If it's `null`, current ord doesn't keep them and you need one of the below.

2. Pin an old ord just for this. Build 0.19.1 (last release with the flag) in a separate `--data-dir`, run `ord --index-sats --index-spent-sats index update`, and query that index. Downside: stale codebase, full sat index from scratch (~hundreds of GB), slow.

3. Compute the ranges yourself. Since you've got a full node (and counterparty-core already in the mix), you can forward-carry ranges directly from raw tx data using the ordinal assignment algorithm — inputs map to outputs in order, FIFO. This is the only approach that scales to arbitrary historical spent outputs without depending on whatever ord retains.

Quick check first: run `ord --version` to be sure it's genuine upstream 0.27.1 and

not a bundled build — that tells us whether option 1's behavior will match upstream.

Want me to sketch the range-tracing logic for option 3, or help you stand up the pinned 0.19.1 index for option 2? If you tell me whether you need *one specific* spent output or *bulk historical* coverage, I can point you at the least painful path.

What does index transactions and addresses do

Synthesized flag functionalities from established documentation >

Both store extra data in ord's own index that it otherwise wouldn't keep:

`--index-transactions` stores every transaction's raw data inside ord's redb index. By default ord doesn't do this — it fetches transactions from Bitcoin Core over RPC on demand (which is why Core needs `-txindex`). Turning it on lets ord serve transaction data (the `/tx` and `/r/tx/<txid>` endpoints, `ord decode`, etc.) straight from its own database without round-tripping to bitcoind. The tradeoff is a big disk cost — you're essentially duplicating the chain's transaction data into the index.

`--index-addresses` builds an address → output mapping so ord can answer "what does this address hold." It powers the `/address/<address>` page and API, returning the UTXOs at an address along with their inscriptions, runes, and sat balance. The key word in the help text is *unspent*: it tracks addresses of the current UTXO set, so it tells you what an address owns **now**, not its full historical activity. (It was added in 0.19.0, which is why your 0.27.1 has it.)

One thing worth flagging given what you're actually after: neither of these helps with spent sat ranges. `--index-addresses` only covers unspent outputs by design, and `--index-transactions` stores tx bytes but doesn't track sat ranges at all — that's still strictly the job of `--index-sats`. So for your spent-output goal, these two are orthogonal; the question remains whether your `--index-sats` build retains spent ranges (option 1 from before) or whether you go the pinned-old-version / manual-trace route.

so does index sat retain spent ranges?

Excavating definitive context about index behavior and performance tradeoffs >

Let me check this against current ord rather than guess — this is the one piece I haven't been able to pin down.

Synthesized historical documentation to definitively resolve spent sat retention question >

No — a plain `--index-sats` index does **not** retain spent ranges. It only keeps sat ranges for the live UTXO set (unspent outputs). When an output gets spent, its ranges are forward-carried into the spending transaction's outputs and the spent output's own entry is dropped.

The history makes this unambiguous. Per casey on the original PR, retaining spent ranges was how the index originally worked, but it was too expensive — taking forever to build and swelling the index to over 10 terabytes. So it was abandoned as the default, and `--index-spent-sats` was added in 0.16 purely as an opt-in to restore that old behavior — with the warning that it would likely require a crazily beefy machine, like 256GB+ of RAM and tens to hundreds of terabytes of disk. Then 0.20 removed even that opt-in. [github](#) [github](#)

You can see the default model in the wild too: when an output is spent, the inputs still have the sat ranges, but the outputs don't — i.e. ord is designed to show ranges only for unspent outputs. [GitHub](#)

So putting the whole thread together for your 0.27.1:

- Plain `--index-sats` → spent outputs come back with `sat_ranges: null`, `spent: true`. No good for your goal.
- No flag exists on 0.27.1 to change that — the opt-in was removed in the 0.20 overhaul, and the prune-by-default behavior is what remains.

That leaves the two paths from before as your only real options: **pin an old ord (0.16–0.19) and build a dedicated index with `--index-spent-sats`** (accepting the brutal disk/RAM cost casey warned about), or **trace the ranges yourself** from raw tx data via the forward-carry algorithm — which, for bulk historical spent outputs, is almost certainly the saner route.

Given you've got counterparty-core and a full node already, do you want me to sketch the forward-carry tracing approach? It's not much code, and it sidesteps the whole 10TB-index problem.

can you link to that conversation by casey

Located and retrieved the GitHub PR discussion link >

Sure — it's pull request #2999, "Add option to retain sat index for spent outputs":

<https://github.com/ordinals/ord/pull/2999>

Casey's comments in that thread are where the cost rationale comes from — that retaining spent ranges was the original index design, abandoned because it ballooned to ~10TB and needed an extreme machine to build, and that the `--index-spent-sats` flag was added as an untested opt-in for anyone wanting to try it. That's also the PR where he notes the rework needed because a spent output could now still carry sat ranges.

If you want the full historical arc, the companion reference is the changelog, where you can watch the flag appear in 0.16.0 and then the index get overhauled in 0.20.0 (#3893 plus the unified UTXO-entry table): <https://github.com/ordinals/ord/blob/master/CHANGELOG.md>

how do I download this entire conversation

Extracting export options and evaluating data download methods >

Let me check the current options for that in Claude's documentation.

Weighed official export options against practical workarounds for single conversation >

A few options, depending on whether you want just this chat or everything:

For this one conversation, readably: the simplest no-extra-tools route is your browser's print dialog — Cmd/Ctrl+P, then "Save as PDF." It captures the visible conversation as you see it. Expand any collapsed sections first so they're included.

Official full export (all your chats): Click your initials in the bottom-left → Settings → Privacy → Export Data. Anthropic emails you a download link (valid ~24 hours; can take minutes to hours depending on volume). The catch is that this dumps *every* conversation as a ZIP of JSON files, not just this one, and JSON isn't very human-readable without a parser. Note it's available on the web and desktop apps but not iOS/Android, and it doesn't include memory data.

Third-party exporters: browser extensions and console scripts exist that export a single conversation to Markdown or PDF with formatting preserved. They work, but they're unofficial and require granting a tool access to your chat content, so weigh that against the convenience — for a one-off, print-to-PDF avoids that tradeoff entirely.

Since these are product details that change, the in-app export steps are worth double-checking at support.claude.com if the menu looks different.

For what it's worth, if your real goal is to keep the ord findings from this thread — the version timeline, the PR rationale, the options for spent sat ranges — I'm happy to write that up as a clean Markdown file you could download directly. Want me to do that?



